Instituto Tecnológico de Costa Rica

Escuela Ingeniería en Computadores

CE - 1104 Fundamentos de Sistemas Computacionales

DOCUMENTACIÓN TÉCNICA

Stranger TEC Morse Translator

Estudiantes:

Mónica Blanco Jiménez

Jehudy Sánchez Gómez

Alexander Rojas Lai

Profesor:
Leonardo Araya

Semestre I 2026

0. Índice....3
1. Introduccion....3
1.1. Objetivos y alcances....3
1.2. Descripcion del Proyecto....3
1.3. Objetivos y alcances.... 3
1.4. Definiciones clave, acrónimos y abreviaturas....4
2. Descripción general del sistema....5
2.1. Perspectiva del producto....6
2.2. Funcionalidades principales....6
2.3. Restricciones del sistema, supuestos y dependencias....7
3. Requisitos del sistema....7
3.1. Requisitos funcionales....7
3.2. Requisitos no funcionales....8
4. Diseño de solución....9
4.1. visión general de la arquitectura....9
4.2. Descripcion de componentes....10
4.3. Stack tecnológico....11
4.4. Diagrama de clases y modelos de datos....11
4.5. Diagrama de flujo y secuencia 12
4.6. Desiciones de diseño y justifiación....13
5. Implementación y validación de calidad....13
5.1. Estructura del repositorio....13
5.2. Entorno de desarrollo, guía de instalación y ejecución....13
5.3. Estrategias de prueba....14
5.4. Resulatdos de pruebas y problemas encontrados....14
6. Declaración de uso de inteligencia artificial....14
6.1. Herramientas de IA utilizadas....14
6.2. Descripcion del uso de inteligencia artificial....15
6.3. Declaración de responsabilidad y autoría....15
7. Conclusiones del proyecto....16
8. Referencias bibliográficas....17

1.Introducción.

1.1. Propósito del documento

Este documento describe el funcionamiento del código morse empleado en sistemas de interfaces gráficas y en circuitos armados con microprocesadores para realizar un juego que traduzca y envíe mensajes entre dicha maqueta que contenga los circuitos necesarios para ejecutar su funcionamiento y la interfaz creada para recibir el mensaje o transmitirlo hacia la maqueta, todo con el fin de aprender a comunicar distintos componentes tanto de software como de hardware mediante el uso de distintas IDE's y librerías. Como objetivos esperamos completar dichas conexiones para expandir nuestros conocimientos en programación e informática, así como aprender a armar circuitos capaces de conectarse a un sistema como el anteriormente descrito mediante wifi. Igualmente, mediante la siguiente documentación esperamos que este proyecto pueda ser de ayuda para otros estudiantes de primer ingreso o que estén por llevar el curso de fundamentos de sistemas computacionales para que usen esta información y nuestro código debidamente documentado y comentado como guía en el caso de que quieran repetir esta modalidad de proyecto con fines de aprendizaje.

1.2. Descripción del proyecto

Durante la ejecución del proyecto se emplearán conocimientos de soldadura, arquitectura de un sistema de código morse mediante un microprocesador raspberry o Arduino, creación de códigos y manejo de GitHub para la realización de dicho juego o proyecto. Dicho sistema con dichas interacciones se clasifica como un Sistema de Interfaz Humano-Maquina con Comunicación Serial y Procesamiento de Eventos. Su aplicación es meramente EdTech por su función de comunicación entre componentes y sistema, y la creación de códigos que requiere para su creación, por lo tanto, el proyecto está hecho meramente con fines educativos, para los integrantes de este mismo o bien para estudiantes de otros niveles que estén interesados en ver su funcionamiento.

1.3. Objetivos y Alcances

Objetivo General

Diseñar un dispositivo empotrado con la capacidad de traducir y transmitir un mensaje mediante código morse, empleando una combinación de hardware y software entre los microprocesadores posibles de emplear en este proyecto y la interfaz gráfica de Python

Objetivos específicos

Brindar utilidad a dos registros de corrimiento para controlar una matriz de 16 LED's, permitiendo la representación de caracteres alfanuméricos en la Raspberry Pi Pico 2 W

Integrar cada periférico de entrada y salida como el caso del buzzer que nos permitirá obtener una retroalimentación auditiva y botones para la captura de los pulsos correspondientes al código morse y ensamblarlos o soldarlos según lo requiera la maqueta para su correcto funcionamiento.

Conseguir que la maqueta mediante el microprocesador empleado sea capaz de conectarse mediante señal bluethoot a otra maqueta, sea de otro estudiante u otra maqueta extra que realicemos con el microprocesador Arduino, si se nos fuera posible debido al tiempo de la realización del proyecto.

Basándonos en estos Alcances esperamos lograr que los dispositivos empotrados y la interfaz gráfica sea capaz de realizar dichas conexiones y funcionar correctamente motivándonos en el surgimiento de las escenas icónicas de la serie en la que se inspira este proyecto Stranger Things, donde la comunicación no verbal fue clave en la trama para que Joyce Byers descubriera la verdad de lo que paso con su hijo, nosotros lo recrearemos con una maqueta de 16 LED's controlados por registros de corrimiento. Abordando desafíos por la implementación de Hardware capaz de comunicarse con el software programado en tiempo real con tal de transmitir un mensaje y la correcta detección de este mismo mediante puntos y rayas o pulsos largos y cortos. Permitiendo así finalmente a estos dos sistemas enfrentarse en una modalidad “Versus” mediante comunicación inalámbrica.

1.4. Definiciones clave, acrónimos y abreviaturas.

GPIO (General Purpose Input/Output): Pines del microcontrolador que funcionan como entradas o salidas.

GUI (Grafical User Interface): Interfaz de usuario.

IDE (Integrated Development Environment): Entorno de desarrollo integrado.

IC: Circuito integrado.

PCB: Placa de circuito impreso.

TCP: Protocolos de transmisión o comunicación de Red.

UART: Protocolo de transmisión de datos entre la Raspberry y la computadora.

VCC: Voltaje de corriente continua.

GND: Tierra o referencia común de potencial cero.

2.Descripción General del Sistema

Se estará trabajando con tres sistemas diferentes y tres pasos para ejecutarlos que van desde:

Crear la maqueta con sus respectivas conexiones correctas en cada nodo para que ningún componente se vea afectado por algún error de manejo del equipo.

Programar el código para que la maqueta interprete las pulsaciones en el botón como el código morse propuesto para el proyecto y mediante un buzzer esta pueda ejecutar sonidos en cada pulsación correspondiente a puntos o líneas.

Mediante el uso de Visual Studio en python con la librería de tkinter diseñar una interfaz gráfica que tenga conexión con la maqueta y su respectivo código para la ejecución del juego.

Seguidamente contamos con algunas restricciones en cuanto a la distancia de viviendas de los integrantes de este grupo por lo que nos estaremos reuniendo en la biblioteca del Tecnológico de Cartago para ejecutar dichos pasos mencionados, otra limitación fue conseguir exactamente la raspberry pi pico W que se nos fue solicitada, sin embargo, al contar con autorización de nuestro docente conseguimos la raspberry pi pico 2 W e igualmente contamos con una placa Arduino de la cual también se tiene

autorización de su uso. Durante la ejecución del proyecto se repartieron las labores de manera equitativa entre el trabajo de los códigos y la construcción de la maqueta, los roles se distribuyeron de la siguiente manera:

Alexander Rojas lideró la conexión de la maqueta y su respectivo diseño.

Jehudy Sánchez lideró la programación del código de la maqueta.

Mónica Blanco lideró la creación de la interfaz gráfica.

2.1. Perspectiva del producto

Este sistema que involucra la comunicación de software y hardware se operará de manera local entre los dispositivos empleando un servidor Flask en python con un API de intermediario para realizar la respectiva comunicación de la maqueta y el equipo que contiene los códigos de la interfaz y de la maqueta. De esta manera se comunicará por un GPIO y un host virtual con el respectivo puerto de nuestra raspberry.

2.2 Funcionalidades principales

Traducción de Mensajes a Código Morse: El sistema convierte frases de texto (de hasta 16 caracteres) en señales de código Morse, representadas mediante pulsos cortos (puntos) y largos (rayas).

Visualización de Caracteres en Panel de LED's: La maqueta muestra el mensaje recibido o ingresado mediante el encendido secuencial de LED’s organizados alfabéticamente en tres filas, emulando la pared de luces de la serie.

Ingreso de Datos mediante Interfaz Física: Permite al usuario introducir caracteres en código Morse utilizando un botón pulsador, donde la duración de la presión define si es un punto o una raya.

Interfaz Gráfica de Usuario (GUI): Aplicación en Python que permite configurar partidas, seleccionar frases aleatorias, visualizar el puntaje en tiempo real y actuar como una de las interfaces de entrada para los jugadores.

Generación de Retroalimentación Auditiva: El sistema utiliza un buzzer pasivo para emitir sonidos que corresponden a la duración de los pulsos Morse enviados, facilitando el control del ritmo por parte del usuario.

Gestión de Modos de Juego y Conectividad: El sistema puede operar en Modo Local (dos jugadores en una misma maqueta vía USB) o Modo Versus (dos maquetas comunicadas inalámbricamente mediante wifi).

Evaluación y Sistema de Puntajes: El software compara la precisión y secuencia del código Morse ingresado por el jugador contra la frase original, asignando puntos según los aciertos.

2.3. restricciones del sistema, supuestos y dependencias

Limitaciones al conseguir el microprocesador que se nos fue solicitado: se tuvo que implementar un microprocesador diferente al pedido por el docente.

Incertidumbre de conexión entre maquetas: se desconoce si se logrará conectar con otras maquetas para la función de versus.

3.Requisitos del Sistema

Sección detallada de los requisitos funcionales y no funcionales para el proyecto.

3.1. Requisitos Funcionales

ID	Descripción	Prioridad	Criterio de Aceptación
RF-001	Panel LED	Alta	Todos los LED se encienden cuando se requiere
RF-002	Conexión con la computadora	Alta	La maqueta se conecta correctamente con

			la interfaz de la computadora
RF-003	Interfaz de usuario	Alta	La interfaz de usuario en la computadora funciona correctamente.
RF-004	Buzzer	Media	El buzzer reproduce correctamente las señales de la computadora
RF-005	Switch de modos	Media	La maqueta es capaz de cambiar entre los dos modos de juego correctamente
RF-006	Conexión con otra maqueta	Baja	La maqueta es capaz de conectarse con otra de su mismo tipo para un modo maqueta versus maqueta

3.2 Requisitos No Funcionales

ID	Categoría	Descripción	Métrica / Valor Esperado

RNF- 01	Rendimiento	El sistema debe responder en ambas direcciones de la maqueta y la interfaz gráfica.	Tiempo de respuesta < 5 segundos debido al rendimiento del hardware del equipo que corra la interfaz
RNF- 02	Escalabilidad	Se espera que el sistema sea capaz de comunicarse con otro sistema igual, pero de distinto puerto.	El rendimiento se reducirá un 10 % al recibir la conexión de otra maqueta
RNF- 03	Usabilidad	El sistema es de fácil entendimiento para otros grupos que no colaboraron en su fabricación	Se espera que el 80% de los usuarios externos que lo ejecuten puedan jugar adecuadamente.

9. Diseño de solución

A continuación, se detallarán las decisiones de diseño tomadas para dar respuesta a los requisitos del sistema. Así como la documentación de la arquitectura, los componentes, el modelo de datos y los flujos principales.

4.1. Visión General de la Arquitectura

Presenta el estilo arquitectónico elegido Cliente-Servidor y Sistemas Empotrados esto debido a la necesidad de separar la lógica de procesamiento pesado y la interfaz de usuario de la gestión directa del hardware. Haciendo que este mismo se divida en dos capas, el host que sería la interfaz en la que el usuario interactúa o recibe las instrucciones y el sistema empotrado donde las interpreta para que nuevamente el host las traduzca. [5]

Esta arquitectura sería la más adecuada debido a que permite ejecutar con certeza las interacciones entre los distintos sistemas empleados en base al código y las librerías usadas en estos. El siguiente diagrama explica más las comunicaciones y funciones de las partes del sistema. [6]

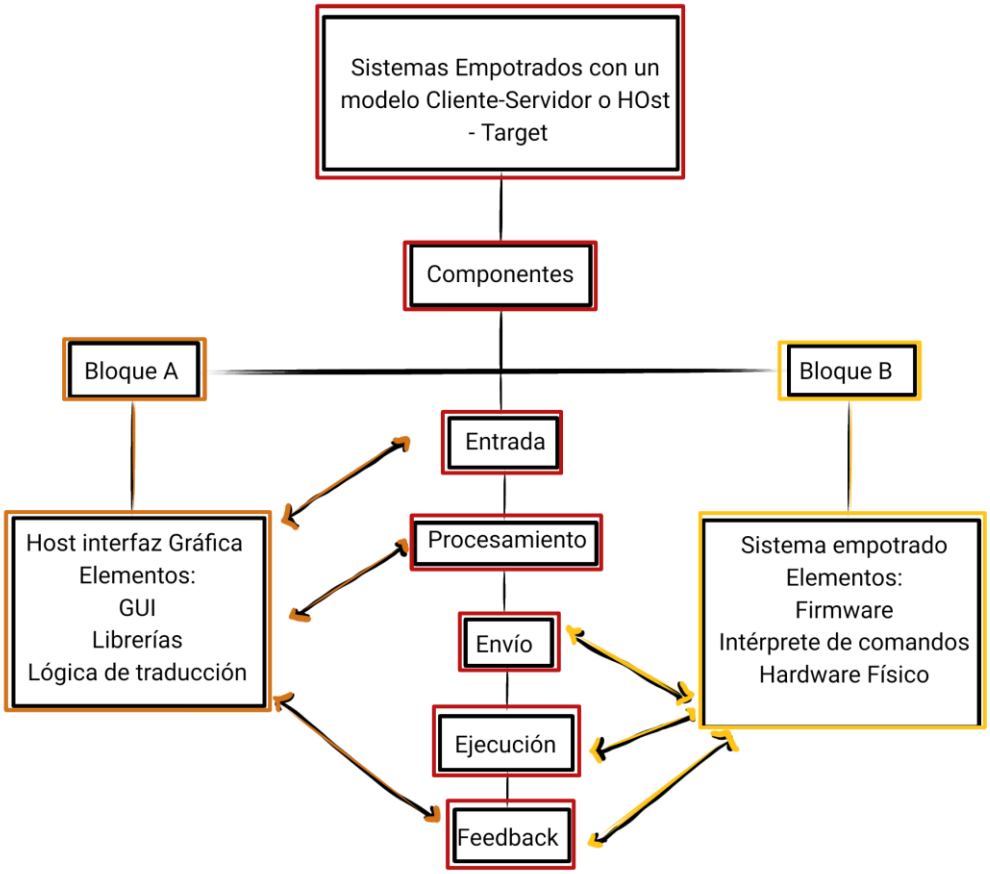

Figura 1. Elaboración propia, diagrama de arquitectura

4.2. Descripción de Componentes

Componente	Responsabilidad	Tecnología	Dependencias
Frontend	Interfaz gráfica que nos da la instrucción	Python 3, Tkinter, random, PySerial	API del backend
Backend, API	Comunicador entre la GUI y el microprocesador	Fast api, python 3	IP y puertos de la raspberry (maqueta)
Código de la maqueta	Configurador de cada componente en la maqueta	Mycropython, Thonny	Backend
Raspberry pi pico 2 w	Microcontrolador de la maqueta	Protocolo TCP/IP sobre wifi	Código de la maqueta
buzzer	Transmisor de sonido para el lenguaje morse	Salida de audio	Código de la maqueta
LED's	Señal visual de letras o símbolos en la maqueta	Registro de corrimientos	Código de la maqueta

Botón	Crea el mensaje de puntos o líneas	Pulsador momentáneo	Código de la maqueta

4.3. Stack Tecnológico

Categoría	Tecnología	Versión	Propósito
Lenguaje (Host)	python	3.13.4	Crear el código de la GUI
Framework GUI	tkinter	8.6.12	Librerías del GUI
Librería de Comunicación	PySerial	3.5	API para comunicar el GUI con el hardware
Lenguaje (Hardware)	Micropython	1.28	Configuración del harware
Microcontrolador	Raspberry pi pico w	2	Controlador de los demás componentes y transmisor de mensaje a la computadora
Salida Visual	Registro de corrimientos	74HC164	Comunicador visual de la maqueta

4.4. Diagrama de Clases y Modelo de Datos

La estructura de clases se ve explicada por el siguiente diagrama en el que todas sus partes tienen relación y comparten funciones

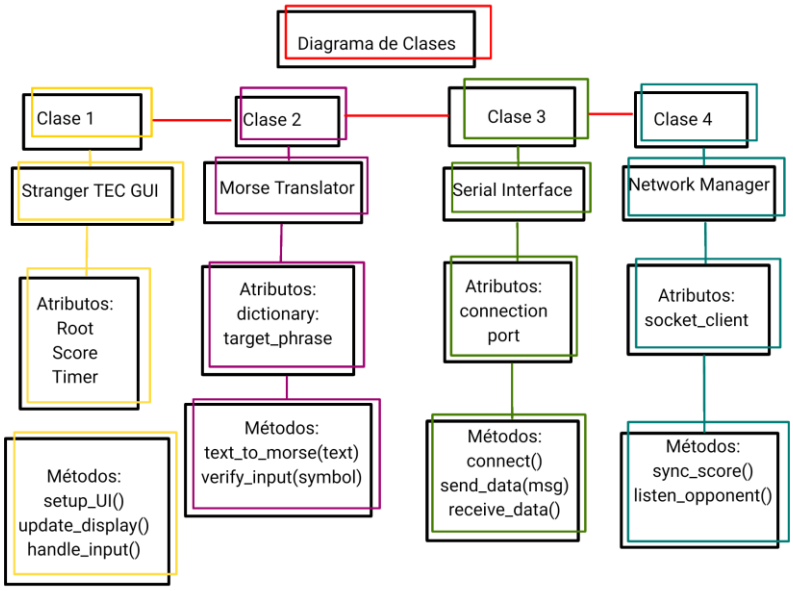

Figura 2. Elaboración propia, diagrama de clases [6]

4.5. Diagrama de Flujo y Secuencia

Mediante el siguiente diagrama se documenta como fluye la información y las secuencias por cada componente hardware y el sistema creado para la maqueta.

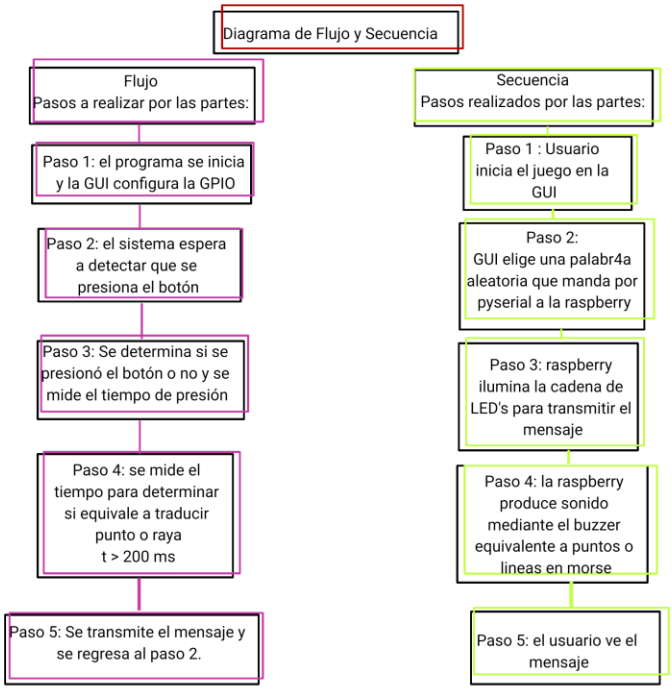

Figura 3. Elaboración propia, diagrama de flujo de informacion [6]

4.6. Decisiones de Diseño y Justificación

Decisión	Alternativas evaluadas	Opción elegida	Justificación}
Arquitectura Cliente servidor	Arquitectura Monolítica	Host-Target	Este sistema separa mejor el procesamiento de la traducción

5. Implementación y validación de calidad

A continuación, se detalla la estructura del repositorio en el que se encuentran adjuntos nuestros códigos y sistemas para que el proyecto resulte funcional en un equipo diferente al que se programó, así como su respectiva guía de instalación.

5.1 Estructura del repositorio

Stranger-Tec_Morse_Translator-Grupo-1 / ------> Nombre del repositorio

|------- docs / ---------> documentación del proyecto

|------ cods / -----------> códigos del proyecto

| |----- host / ----------> códigos de la PC

| | |------ GUI / ------------> interfaz gráfica

| | |-------Fondo/ -----------> imagen de fondo de la GUI

| |----- semp / ------------------> sistema empotrado

| | |------Raspcode / ----------------> código de la maqueta y raspberry

5.2 Entorno de Desarrollo, Guía de instalación y Ejecución

Entorno:

- Sistema operativo: Windows 11
- Lenguajes para usados: python y micropython
- IDE’s requeridos: Visual Studio Code y Tonny
- Bibliotecas necesarias: tkinter, pillow

Guía de instalación y ejecución

1. Descargar el repositorio.
2. Abrir los códigos en su respectiva IDE, el nombrado GUI con visual Studio python y el nombrado Raspcode con Tonny Micropython.
3. Instalar las librerías tkinter (pip install tkinter) y pillow (pip install PIL)
4. Verificar que la imagen nombrada Fondo.jpeg este en el mismo directorio para que se ejecute.
5. Ejecutar los códigos y la maqueta emitirá el sonido dependiendo del modo de juego.

5.3. Estrategias de prueba

No se ejecutaron casos de prueba para este proyecto

5.4 Resultados de pruebas y problemas encontrados

No se logró que la API funcionara como intermediario pues no se logró crear el servidor, se optó por la alternativa de crear un socket que es parte de una API que permite poner el puerto único de la raspberry y esta estará conectada automáticamente a la GUI. Por lo tanto, no es posible conectar la maqueta a otras mediante uso de wifi.

6. Declaración de uso de inteligencia artificial

En este apartado se mencionan las herramientas de inteligencia artificial empleadas en el proyecto y se detalla con que fines estas fueron utilizadas.

6.1 Herramientas de IA utilizadas

Herramienta de IA	Tipo	Período de uso	Propósito
Blackbox IA	Asistente de código	25 de abril 2026	Ideas para lógica del código
Gemini	Modelo de lenguaje	20 de abril 2026	Ideas de redacción, investigación para los diagramas e investigación de cómo funciona una API y su concepto.
Claude	Modelo de lenguaje	25 de abril 2026	Mejor búsqueda de información para la

Codex	Modelo de lenguaje	20 de abril 2026	Explicación de uso de código para la raspberry

6.2. Descripción del Uso de Inteligencia Artificial

Black Box AI: se utilizó en el código de la interfaz gráfica para revisión de indentación en la línea 202 y para buscar ideas de lógica en las líneas 132 y 112. Así como los directorios en los que se encuentran los colores y diseños o relieves compatibles o disponibles a usar con tkinter.

Gemini: se utilizó para la redacción del documento en el apartado de clases y diagramas de la estructura y arquitectura del sistema diseñado, pues no se tenía claro como clasificarlo, así mismo se empleó para investigación respecto a las API's que asignó ciertos enlaces de ayuda que se encuentran en las referencias bibliográficas de este documento.

Claude: Se utilizó como apoyo para la decoración de la interfaz gráfica, la resolución de conflictos entre gestores de geometría de tkinter y la implementación de un canvas como contenedor principal.

6.3 Declaración de responsabilidad y autoría

Los suscritos, integrantes del equipo de trabajo del proyecto "Stranger Tec Morse Translator", declaramos que:

1. El diseño, análisis, desarrollo y conclusiones del presente proyecto son resultado de nuestro trabajo intelectual y esfuerzo como equipo.

2. Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto de nuestro trabajo.

3. Todo el contenido generado por IA fue revisado, comprendido, validado y, cuando fue necesario, corregido por los integrantes del equipo.

4. Asumimos plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.

5. El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

Firma de los integrantes del grupo:

Mónica Blanco J

Jehudy Sánchez Gómez

Alexander Pengyang Rojas Lai

7. Conclusiones del proyecto

Se logró aprender a programar un sistema empotrado y servidor-cliente mediante el uso de librerías de tkinter y lenguajes de programación como python y micropython, usando parte de una API como intermediaria.

Carecemos del tiempo suficiente para para completar dicha API y concluir con uno de los objetivos referentes al “modo vs” que resultaba opcional en este proyecto. En cambio, la maqueta y la GUI se conectaron mediante un socket por la IP proveniente de fábrica en la raspberry igualmente empleando Py serial en el GUI para realizar dicha conexión.

No obstante, los demás objetivos planteados se lograron con éxito, la maqueta tiene conexión al GUI, se programó según las indicaciones del docente y los modos de juego se ejecutan satisfactoriamente.

Así mismo, nosotros los integrantes del grupo nos llevamos bastante conocimiento respecto a los sistemas empotrados pues se desconocía como tal como lograr dicha conexión. Además de que desconocíamos el concepto de una API y como crear un servidor. Sí bien no llegamos a completarla, conocer su mera existencia nos permitirá desarrollar mejores proyectos en el futuro.

Como recomendaciones para alguien interesado en desarrollar algún proyecto de este tipo, se le dejaran referencias de la información investigada que le será de utilidad, donde podrá encontrar herramientas como bibliotecas para desarrollar su programa.

8. Referencias bibliográficas

[1] Raspberry Pi Foundation, “Raspberry Pi Pico W Datasheet: An RP2040-based microcontroller board with wireless stack,” Raspberry Pi Documentation, 2022. [En línea]. Disponible en: https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf

[2] Texas Instruments, “74HC164; 74HCT164 - 8-bit serial-in, parallel-out shift register datasheet,” Rev. 4, 2021. [En línea]. Disponible en: https://www.ti.com/lit/ds/symlink/sn74hc164.pdf

[3] Python Software Foundation, “Python Language Reference, version 3.12,” 2024. [En línea]. Disponible en: https://docs.python.org/3/

[4] M. MicroPython, “MicroPython Documentation: Library Reference,” 2026. [En línea]. Disponible en: https://docs.micropython.org/en/latest/

[5] G. Coulouris, J. Dollimore, T. Kindberg, y G. Blair, Distributed Systems: Concepts and Design, 5ta ed. Boston, MA, USA: Addison-Wesley, 2011. (Referencia para sustentar la arquitectura Cliente-Servidor).

[6] P. Marwedel, Embedded System Design: Embedded Systems Foundations of Cyber-Physical Systems, and the Internet of Things, 3ra ed. Dortmund, Alemania: Springer Nature, 2018. (Referencia para sustentar el diseño de sistemas empotrados).

[7] International Morse Code, “Recommendation ITU-R M.1677-1: International Morse code,” International Telecommunication Union, Ginebra, Suiza, 2009.

[9] Python Software Foundation, “Socket — Low-level networking interface,” Python 3.12 Documentation, 2026. [En línea]. Disponible en: https://docs.python.org/3/library/socket.html

[10] R. Fielding y J. Reschke, “Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content,” RFC 7231, Internet Engineering Task Force (IETF), 2014. (Ideal si tu API usa protocolos web para enviar los mensajes Morse).

[11] Python Software Foundation, “tkinter — Python interface to Tcl/Tk: Canvas,” *Python Documentation*, 2025. [Online]. Available: https://docs.python.org/3/library/tkinter.html#tkinter.Canvas. [Accessed:

[12] Read the Docs, “Pillow Documentation,” *Pillow Docs*. [Online]. Available: https://pillow.readthedocs.io.

[13] Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” *Python Documentation*. [Online]. Available: https://docs.python.org/3/library/tkinter.html.

[14] M. Roseman, “TkDocs Tutorial,” *TkDocs*. [Online]. Available: https://tkdocs.com.

[15] Raspberry Pi Ltd, “Raspberry Pi Documentation — Computers,” *Raspberry Pi Documentation*. [Online]. Available: https://www.raspberrypi.com/documentation/computers/raspberry-pi.html

[16] Stack Exchange Inc., “Wiring diagram software,” *Raspberry Pi Stack Exchange*. [Online]. Available: https://raspberrypi.stackexchange.com/questions/42735/wiring-diagram-software.